

Introduction to Programming with Python

Contents

1	What is Software?	3
2	How Do We Write Instructions?	3
2.1	Types of Programming Languages	3
2.1.1	1. Machine Level Language	3
2.1.2	2. Assembly Language	4
2.1.3	3. High Level Languages	4
3	Important Conclusion	4
4	Assembler, Compiler and Interpreter	5
4.1	Assembler	5
4.2	Compiler	5
4.3	Interpreter	5
5	How Python Executes Code	5
6	Different Types of Python	6
6.1	CPython	6
6.2	Jython	6
6.3	IronPython	6
6.4	PyPy	6
7	What Actually is Programming?	7
7.1	Data Processing	7
7.2	Data Transfer	7
8	Data Types in Python	7
8.1	Integer	7
8.2	Floating Point Numbers	8
8.3	String	8
8.4	Boolean	8
8.5	NoneType	9
9	Variables	9
10	Constants	9

11 Building a Small Program Step by Step	9
11.1 Step 1: Store Numbers	9
11.2 Step 2: Add Them	10
12 Input Function	10
13 Unexpected Problem	10
14 Type Conversion	11
15 Final Program	11
16 Final Thoughts	12

1 What is Software?

Software is a combination of small programs.

And what is a program?

A program is a combination of computer instructions.

Just like humans follow instructions:

- Wake up
- Brush teeth
- Go to college

Similarly, computers also follow instructions:

- Take input
- Add numbers
- Show result

A computer never becomes “smart” magically. It only follows instructions written by programmers.

So basically:

Software $\rightarrow$ Programs $\rightarrow$ Instructions

2 How Do We Write Instructions?

Computers understand only one language: **Machine Language**

But humans cannot easily write in machine language because it looks scary like this:

```
10101010 11001100 11110000
```

Imagine writing Facebook, YouTube, or games using only 0s and 1s. You would probably retire before finishing the login page.

So different programming languages were created.

2.1 Types of Programming Languages

2.1.1 1. Machine Level Language

This is the lowest level language.

- Written in binary (0 and 1)
- Directly understood by computer
- Very fast
- Very hard for humans

Example:

```
10110000 01100001
```

2.1.2 2. Assembly Language

A little easier than machine language.

Instead of 0s and 1s, small words are used.

Examples:

- MOV
- ADD
- SUB

Example:

```
MOV A, 10  
ADD A, 20
```

Still difficult for large software development.

2.1.3 3. High Level Languages

These are human-friendly languages.

Examples:

- Python
- Java
- C
- C++
- JavaScript

Example in Python:

```
print("Hello World")
```

Much easier, right?

3 Important Conclusion

As programming languages become more high-level:

- Easier for humans
- Harder for computers

Why?

Because computers still understand only machine language.

So somebody has to translate our code into machine language.

That “somebody” is:

- Assembler
- Compiler
- Interpreter

4 Assembler, Compiler and Interpreter

4.1 Assembler

Converts Assembly Language into Machine Language.

Assembly Code $\rightarrow$ Machine Code

4.2 Compiler

Converts the whole program into machine code before execution.

Examples:

- C
- C++

Advantages:

- Fast execution

Disadvantages:

- Need compilation step

4.3 Interpreter

Executes code line by line.

Examples:

- Python
- JavaScript

Advantages:

- Easier debugging

Disadvantages:

- Slower than compiled languages

5 How Python Executes Code

Many students think Python is only interpreted.

Actually, Python uses both:

- Compiler
- Interpreter

Here is what happens:

1. Python source code (`.py`) is first compiled into bytecode
2. Bytecode is stored in `.pyc` files
3. Python Virtual Machine (PVM) interprets the bytecode

So internally:

Python Code $\rightarrow$ Bytecode $\rightarrow$ PVM Execution

Sneaky Python. It uses both worlds.

6 Different Types of Python

Python has different implementations.

6.1 CPython

This is the standard and most popular Python.

- Written in C language
- Created by Guido van Rossum
- Most Python users use this

When people say “Python”, they usually mean CPython.

6.2 Jython

- Python for Java platform
- Runs on JVM (Java Virtual Machine)

Useful when working with Java applications.

6.3 IronPython

- Python for .NET platform
- Developed by Microsoft technologies

6.4 PyPy

- Faster Python implementation
- Uses JIT (Just In Time compilation)

Designed mainly for performance improvement.

7 What Actually is Programming?

Programming is basically:

- Data Processing
- Data Transfer

7.1 Data Processing

Taking data and changing it into something useful.

Example:

- Calculating marks
- Editing images
- Banking calculations

Input:

```
50 + 20
```

Processed Result:

```
70
```

7.2 Data Transfer

Moving data from one place to another.

Examples:

- Sending message in WhatsApp
- Uploading photos
- Watching YouTube videos

Your message travels from your device to another device.
That is data transfer.

8 Data Types in Python

Before programming, we need to understand data.

Different kinds of data exist.

8.1 Integer

Whole numbers.

Examples:

```
10  
-5  
1000
```

8.2 Floating Point Numbers

Decimal numbers.

Examples:

```
3.14
99.99
-0.5
```

8.3 String

Text data.

Examples:

```
"Hello "
'Python '
"""Multi line string"""
```

Three ways to write strings:

- Double quotes
- Single quotes
- Triple quotes

Triple quotes are useful for multi-line text.

8.4 Boolean

Represents only two values:

- True
- False

Example:

```
is_logged_in = True
```

Why does it exist?

Because many situations are yes/no type:

- Is user logged in?
- Is payment completed?
- Is student passed?

8.5 NoneType

Represents nothing or absence of value.

Example:

```
data = None
```

Why does it exist?

Sometimes value is not available yet.

Example:

- User has not uploaded profile picture
- Student marks not entered yet

9 Variables

Programs need temporary storage.

That storage is called a variable.

Example:

```
a = 10
```

Here:

- `a` is variable name
- `10` is stored data

Variables help us store data during program execution.

10 Constants

Sometimes values should not change.

By convention, constants are written in UPPERCASE.

Example:

```
PI = 3.14  
MAX_USERS = 100
```

Python does not strictly stop changes.

It is just a programmer convention saying:

“Dear other programmers, please don’t change this.”

11 Building a Small Program Step by Step

11.1 Step 1: Store Numbers

```
a = 10  
b = 20
```

11.2 Step 2: Add Them

```
a = 10
b = 20

c = a + b

print(c)
```

Output:

```
30
```

Simple enough.

But there is a problem.

Every time we want different numbers, we must edit the code manually.

Very annoying.

12 Input Function

So Python gives us input.

```
a = input()
```

Now user can give data.

But there is another problem.

The user sees nothing on screen and becomes confused:

“Did program hang? Did laptop die? Did universe collapse?”

Actually the program is waiting for input.

So we give message to user.

```
a = input("Enter a number: ")
```

Much better.

13 Unexpected Problem

Now try this:

```
a = input("Enter first number: ")
b = input("Enter second number: ")

c = a + b

print(c)
```

Input:

```
10
20
```

Output:

```
1020
```

Wait...

That is not addition.

That is **string concatenation**.

Because `input()` always returns string data.

So:

```
"10" + "20"
```

becomes:

```
"1020"
```

14 Type Conversion

We need to convert string into integer.

```
int("10")
```

Now Python treats it as number.

15 Final Program

```
a = int(input("Enter a number: "))
# a = 60

b = int(input("Enter another number: "))
# b = 30

c = a * b
d = a / b
z = a + b

print(z)
print(c)
print(d)
```

Example Output:

```
Enter a number: 60
Enter another number: 30

90
1800
2.0
```

16 Final Thoughts

Programming is not magic.

It starts from:

- Data
- Variables
- Instructions
- Logic

Every huge software system eventually becomes combinations of these small concepts.

Even giant applications started from small programs like:

```
print("Hello World")
```

So yes, today you learned basics.

Tomorrow? Maybe your own software company.